EPISODE 2

React, Frontend Architecture & Next.js

From Components to Production-Ready Apps

React Fundamentals	Components & Props
useState & Re-rendering	useEffect Lifecycle
Custom Hooks	React Router
TanStack Query	Context API & Zustand
Redux Toolkit	React Hook Form & Zod
Performance Optimization	Next.js Full Framework

TABLE OF CONTENTS

- Zod schema
- Controlled vs uncontrolled

12 **Performance Optimization**

- React.memo, useMemo, useCallback
- Code splitting
- DevTools

13 **Introduction to Next.js**

- App Router
- File-based routing
- Layouts

14 **Rendering Strategies**

- CSR, SSR, SSG, ISR
- When to use what

15 **APIs & Server Actions**

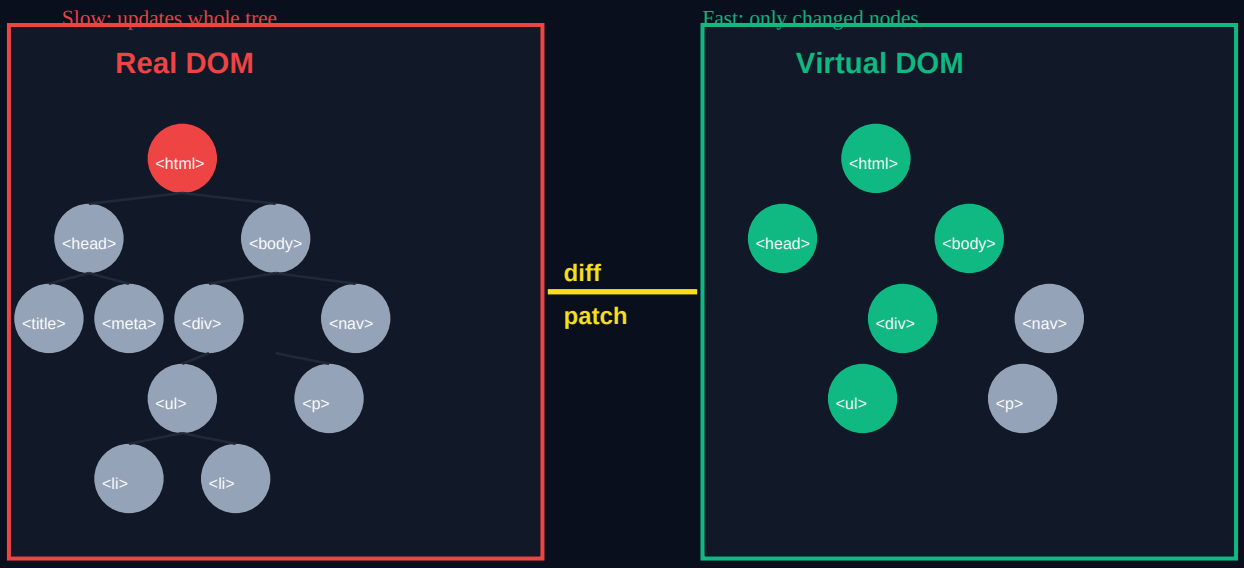
- Route handlers
- Server Actions
- DB integration

CHAPTER 1 — WHY REACT EXISTS

Problems React Solves

Before React (2013), building interactive UIs meant manually updating the DOM — querySelector, innerHTML, addEventListener — for every single state change. With hundreds of UI states, this became a nightmare to maintain. React introduced a DECLARATIVE approach: you describe what the UI should look like, React figures out how to update the DOM.

Real DOM vs Virtual DOM



Virtual DOM: React diffs the virtual tree, then patches only changed nodes in the real DOM

How the Virtual DOM Works

1. State changes → React creates a new Virtual DOM tree (just a JS object).
2. React DIFFS the new tree against the previous one (reconciliation algorithm).
3. React calculates the MINIMUM number of real DOM operations needed.
4. Only changed nodes are updated in the actual browser DOM.

Result: Far fewer expensive real DOM operations = faster UI updates.

Approach	Imperative (jQuery era)	Declarative (React)
Focus	How to update the DOM	What the UI should look like
Code	Manual DOM manipulation	Describe state → React updates
Bugs	Easy to miss updates	UI always reflects state
Scale	Spaghetti at 1000+ elements	Component tree scales cleanly
Example	el.style.display = 'none'	if (isVisible) return

SPA — Single Page Application

Traditional sites: every click loads a NEW HTML page from the server (full reload).

SPAs: One HTML file loaded once. JavaScript handles ALL navigation client-side.

React swaps components in/out — no page reload, instant transitions.

The browser URL changes (React Router), but NO server request is made.

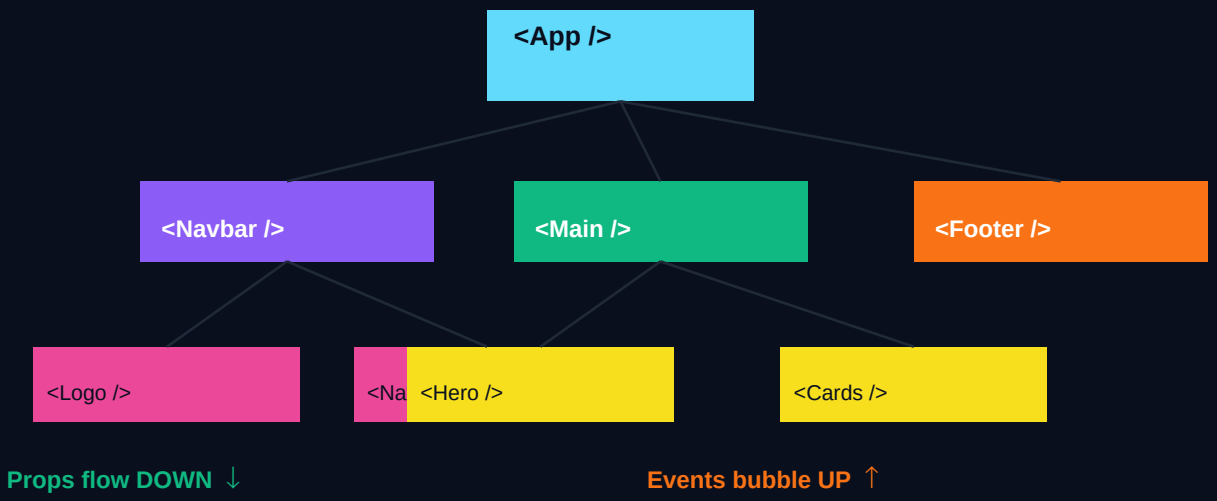
Benefit: App-like experience, faster navigation, no white-flash between pages.

Trade-off: Larger initial JS bundle, SEO complexity (solved by Next.js SSR).

CHAPTER 2 — COMPONENTS & PROPS

React Components

A component is a reusable, self-contained piece of UI. Think of it like a LEGO brick — you create small, focused components and compose them into complex UIs. Every React app is a tree of components.



Component Tree: App is the root, broken into smaller focused components

```
// Functional Component — the standard in modern React
function UserCard({ name, role, avatar, isOnline }) {
  return (
    <div className='card'>
      <img src={avatar} alt={name} />
      <h2>{name}</h2>
      <p>{role}</p>
      <span className={isOnline ? 'online' : 'offline'}>
        {isOnline ? '■ Online' : '■ Offline'}
      </span>
    </div>
  );
}

// Usage — passing props
<UserCard
  name="Alice Johnson"
  role="Senior Developer"
  avatar="/avatars/alice.jpg"
  isOnline={true}
/>
```

Rendering Lists with map() & Keys

```
const users = [
  { id: 1, name: 'Alice', role: 'Dev' },
  { id: 2, name: 'Bob',   role: 'Design' },
  { id: 3, name: 'Carol', role: 'PM' },
];

function UserList() {
  return (
    <ul>
      {users.map(user => (
        // key must be unique & stable — don't use array index!
        <li key={user.id}>
          <UserCard {...user} />  {/* spread props */}
        </li>
      ))}
    </ul>
  );
}
```

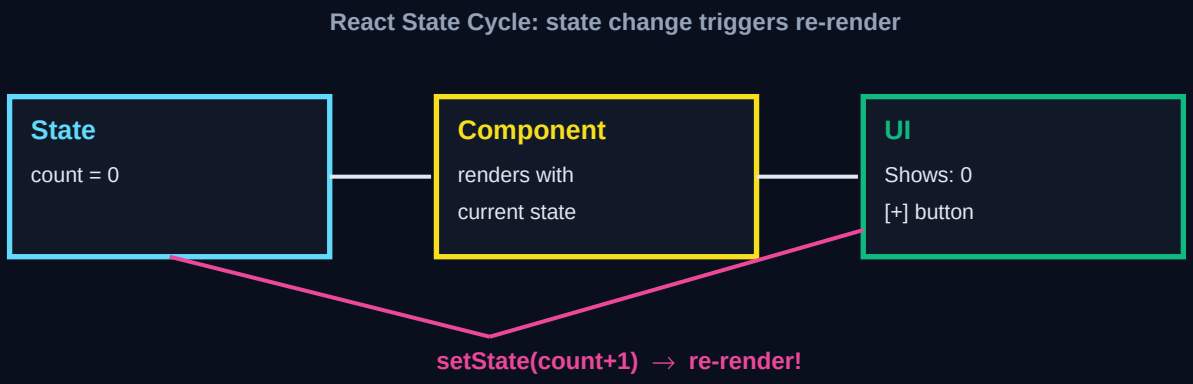
Props — Key Rules

- Props flow ONE WAY: parent → child (never child → parent directly).
- Props are READ-ONLY inside the child — never mutate props!
- Use default values: function Btn({ color = 'blue', size = 'md' }) {}
- PropTypes or TypeScript for type safety (TypeScript preferred).
- To send data UP, pass a callback function as a prop: onSubmit={handleSubmit}.
- Spread operator: <Component {...props} /> passes all props at once.

CHAPTER 3 — STATE & RE-RENDERING

useState Hook

State is data that changes over time, triggering UI updates. When state changes, React re-renders the component (and its children) to reflect the new state. useState is the fundamental hook for local component state.



State cycle: user action → setState → React re-renders → updated UI

```
import { useState } from 'react';

function Counter() {
  // [currentValue, setterFunction] = useState(initialValue)
  const [count, setCount] = useState(0);
  const [step,  setStep]  = useState(1);

  const increment = () => setCount(prev => prev + step); // use callback form!
  const decrement = () => setCount(prev => prev - step);
  const reset     = () => setCount(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={increment}>+{step}</button>
      <button onClick={decrement}>-{step}</button>
      <button onClick={reset}>Reset</button>
      <input type='number' value={step}
             onChange={e => setStep(Number(e.target.value))} />
    </div>
  );
}
```

useState — Critical Rules

■ NEVER mutate state directly: state.items.push(x) — this BREAKS React!

✓ Always create new value: setItems([...items, newItem])

■ setState is ASYNC — don't read state immediately after setting it.

✓ Use functional update: setState(prev => prev + 1) when depending on old value.

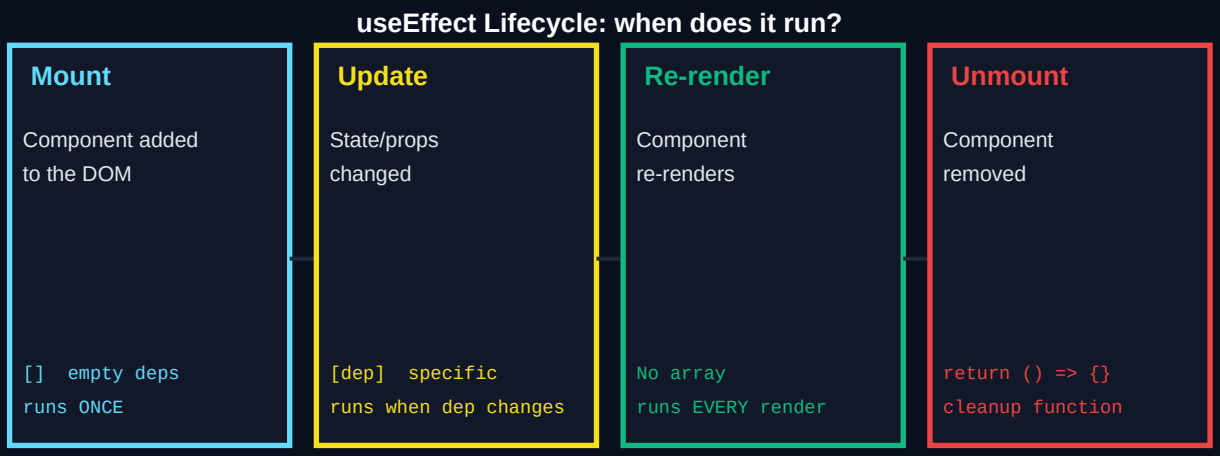
■ Object/array state: spread to create new reference: setUser({...user, name: 'X'})

✓ Multiple state = multiple useState calls (or useReducer for complex state).

CHAPTER 4 — REACT LIFECYCLE & useEffect

useEffect — Side Effects in React

useEffect lets you synchronize your component with external systems — APIs, subscriptions, timers, browser APIs. It runs AFTER the render is committed to the DOM. Understanding the dependency array is crucial to avoiding infinite loops and bugs.



useEffect: behavior changes based on the dependency array you provide

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser]       = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    // Runs when userId changes
    let cancelled = false; // prevent stale updates

    async function fetchUser() {
      try {
        setLoading(true);
        const res  = await fetch(`/api/users/${userId}`);
        const data = await res.json();
        if (!cancelled) setUser(data);
      } catch (err) {
        if (!cancelled) setError(err.message);
      } finally {
        if (!cancelled) setLoading(false);
      }
    }

    fetchUser();
    return () => { cancelled = true; }; // Cleanup!
  }, [userId]); // re-run when userId changes

  if (loading) return <Spinner />;
  if (error)   return <Error message={error} />;
  return <div>{user?.name}</div>;
}
```

CHAPTER 5 — EVENT HANDLING & CONDITIONAL RENDERING

Controlled Inputs & Events

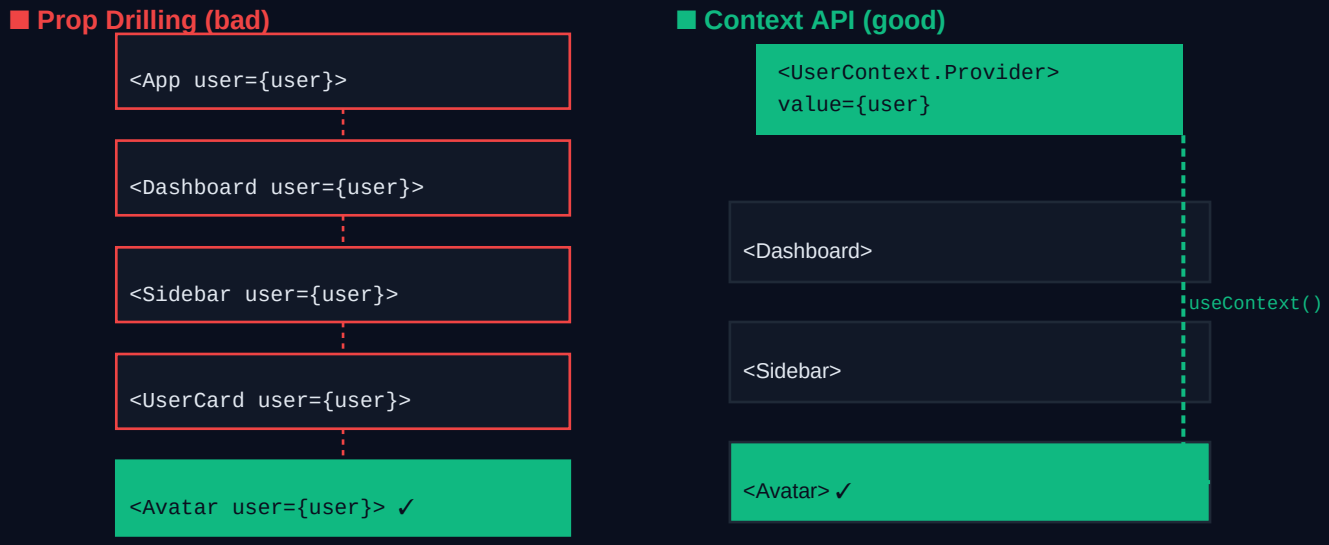
```
function LoginForm() {
  const [email,    setEmail]    = useState('');
  const [password, setPassword] = useState('');
  const [errors,   setErrors]   = useState({});

  const validate = () => {
    const errs = {};
    if (!email.includes('@')) errs.email = 'Invalid email';
    if (password.length < 8)  errs.password = 'Min 8 characters';
    return errs;
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // stop page reload
    const errs = validate();
    if (Object.keys(errs).length) { setErrors(errs); return; }
    // Submit...
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)}
             placeholder='Email' />
      {errors.email && <p className='error'>{errors.email}</p>}
      <input type='password' value={password}
             onChange={e => setPassword(e.target.value)} />
      {errors.password && <p>{errors.password}</p>}
      <button type='submit'>Login</button>
    </form>
  );
}
```

Conditional Rendering Patterns

```
// Pattern 1: && operator (short-circuit)
{isLoggedIn && <UserMenu />}
{error && <ErrorBanner message={error} />}

// Pattern 2: Ternary operator
{isLoading ? <Spinner /> : <Content data={data} />}

// Pattern 3: Early return
function Component({ user }) {
  if (!user) return null;  // render nothing
  if (user.banned) return <BannedMessage />;
  return <UserDashboard user={user} />;
}

// Pattern 4: Object map (best for many conditions)
const STATUS_COMPONENTS = {
  loading: <Spinner />,
  error:   <ErrorState />,
  empty:   <EmptyState />,
  success: <DataTable />,
};
return STATUS_COMPONENTS[status] ?? null;
```

CHAPTER 6 — COMPONENT ARCHITECTURE

Prop Drilling & the Context Solution



Prop drilling passes data through every level; Context teleports it directly

```
// Creating Context
const UserContext = createContext(null);

// Provider wraps the tree
function App() {
  const [user, setUser] = useState(null);
  return (
    <UserContext.Provider value={{ user, setUser }}>
      <Dashboard />  {/* no props needed! */}
    </UserContext.Provider>
  );
}

// Any descendant can consume directly
function Avatar() {
  const { user } = useContext(UserContext); // no props!
  return <img src={user.avatar} alt={user.name} />;
}
```

Pattern	Smart Component	Dumb Component
Also called	Container / Page	Presentational / UI
Handles	State, API calls, logic	Only renders from props
Has state	Yes	Rarely
Reusable	Less reusable	Highly reusable
Example	<UserPage />	<UserCard />
Tests	Integration tests	Snapshot / unit tests

CHAPTER 7 — REACT HOOKS DEEP DIVE

All Core Hooks Explained

React's built-in hooks — each solves a specific problem

```
// useRef — DOM access & mutable values (doesn't trigger re-render)
function VideoPlayer() {
  const videoRef = useRef(null);
  const play = () => videoRef.current.play();
  return <video ref={videoRef} src='/video.mp4' />;
}

// useMemo — expensive calculation memoization
const sortedUsers = useMemo(
  () => users.slice().sort((a, b) => a.name.localeCompare(b.name)),
  [users] // only recalculate when users changes
);

// useCallback — stable function reference
const handleDelete = useCallback((id) => {
  setUsers(prev => prev.filter(u => u.id !== id));
}, []); // stable — won't re-create on every render

// Custom Hook — extracting reusable logic
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(
    () => JSON.parse(localStorage.getItem(key)) ?? initialValue
  );
  const set = (val) => {
    setValue(val);
    localStorage.setItem(key, JSON.stringify(val));
  };
  return [value, set];
}

// Usage: replaces useState + localStorage boilerplate
const [theme, setTheme] = useLocalStorage('theme', 'dark');
```

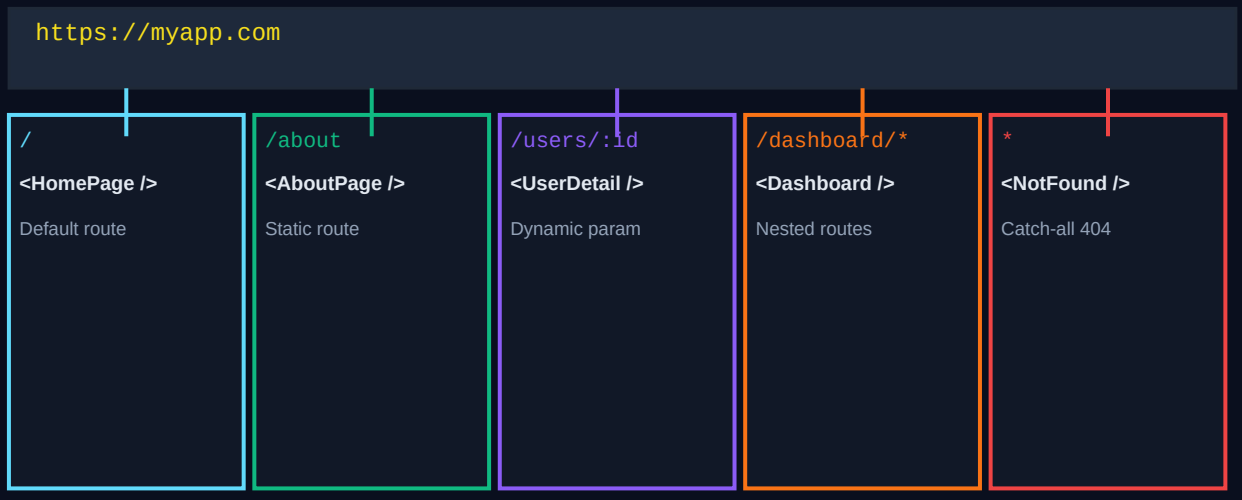
Rules of Hooks — Never Break These

1. Only call hooks at the TOP LEVEL — never inside loops, conditions, or nested functions.
2. Only call hooks from REACT FUNCTIONS — not regular JS functions.
3. Custom hooks MUST start with 'use' — (useMyHook, not myHook).
4. The order of hook calls must be the same every render (React tracks by order).

WHY: React relies on call order to associate hook state with the right component instance.

CHAPTER 8 — ROUTING WITH REACT ROUTER

React Router v6



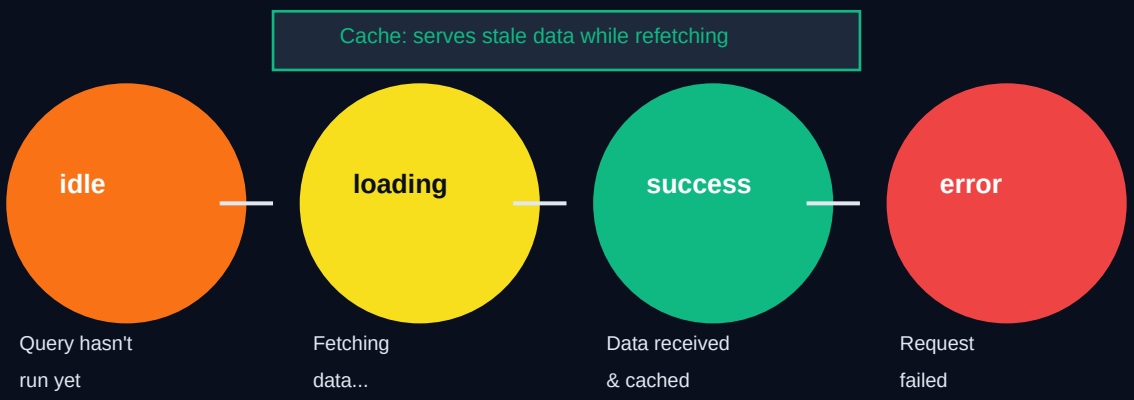
React Router: URL-to-component mapping, all handled client-side

```
import { BrowserRouter, Routes, Route, Link, useParams, useNavigate } from 'react-router-dom';

// App.jsx — Route definitions
function App() {
  return (
    <BrowserRouter>
      <Navbar />  {/* always visible */}
      <Routes>
        <Route path='/'            element={<Home />} />
        <Route path='/users'       element={<UserList />} />
        <Route path='/users/:id'   element={<UserDetail />} />
        <Route path='/dashboard'   element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
        <Route path='*'            element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

// Dynamic Route — reading URL params
function UserDetail() {
  const { id } = useParams();  // gets ':id' from URL
  const navigate = useNavigate();
  // ...
  return (
    <div>
      <button onClick={() => navigate(-1)}>← Back</button>
      <button onClick={() => navigate('/users')}>All Users</button>
    </div>
  );
}
```

CHAPTER 9 — SERVER STATE & TANSTACK QUERY

Why TanStack Query?

Manually managing server state (loading, error, refetch, cache) with useState + useEffect leads to complex, buggy code. TanStack Query (formerly React Query) handles all of this automatically: caching, background refetching, deduplication, and stale-while-revalidate.



TanStack Query: automatic caching, background refetch, deduplication

TanStack Query: automatic cache management with stale-while-revalidate strategy

```
import { QueryClient, QueryClientProvider, useQuery, useMutation } from '@tanstack/react-query';

// Setup — wrap app with provider
const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000 } }, // cache 1 min
});
// <QueryClientProvider client={queryClient}><App /></QueryClientProvider>

// useQuery — fetching data
function UserList() {
  const { data, isLoading, isError, error, refetch } = useQuery({
    queryKey: ['users'],          // cache key — array identifies this query
    queryFn: () => fetch('/api/users').then(r => r.json()),
    staleTime: 5 * 60 * 1000,    // consider fresh for 5 minutes
    retry: 3,                     // retry failed requests 3 times
  });

  if (isLoading) return <Spinner />;
  if (isError)   return <p>Error: {error.message}</p>;
  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

// useMutation — creating/updating/deleting data
function CreateUser() {
  const mutation = useMutation({
    mutationFn: (newUser) => fetch('/api/users', {
      method: 'POST',
      body: JSON.stringify(newUser),
    }),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['users'] }); // refresh list!
    },
  });

  return <button onClick={() => mutation.mutate({ name: 'Alice' })}>
    {mutation.isPending ? 'Saving...' : 'Create User'}
  </button>;
}
```

CHAPTER 10 — GLOBAL STATE MANAGEMENT

Zustand vs Redux Toolkit

Zustand

✓ ~1KB bundle size
✓ Zero boilerplate
✓ No Provider needed
✓ Direct state mutation
✓ Outside React ok
Simple slice = store

Redux Toolkit

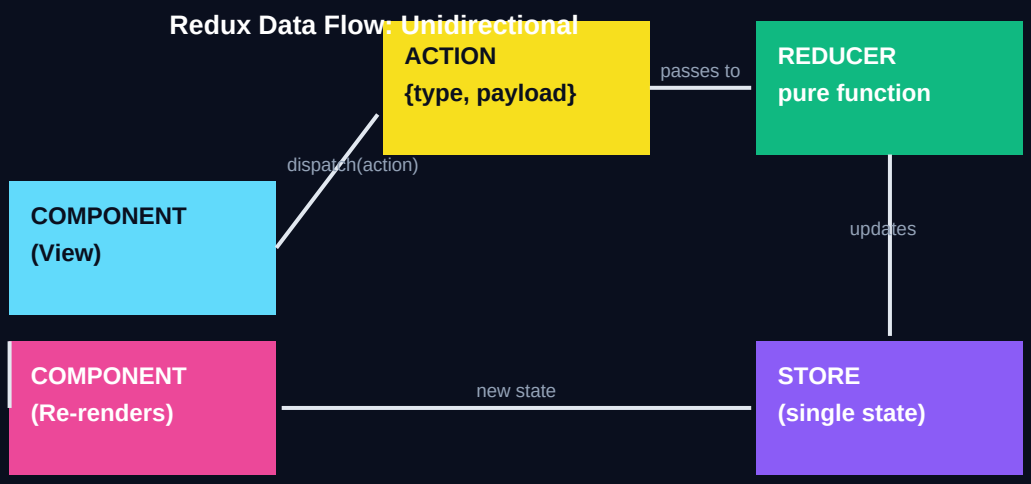
✓ DevTools (time travel)
✓ Predictable updates
✓ Large team scaling
✗ More boilerplate
✗ Larger bundle
createSlice() helps

Choose based on team size, complexity, and DevTools needs

Zustand — Simple & Minimal

```
import { create } from 'zustand';

// Define store — state + actions in one place
const useCartStore = create((set, get) => ({
  items: [],
  total: 0,

  addItem: (product) => set((state) => ({
    items: [...state.items, product],
    total: state.total + product.price,
  })),

  removeItem: (id) => set((state) => ({
    items: state.items.filter(i => i.id !== id),
    total: state.items.filter(i => i.id !== id).reduce((s,i) => s+i.price, 0),
  })),

  clearCart: () => set({ items: [], total: 0 }),
}));

// Use in ANY component — no Provider needed!
function CartIcon() {
  const { items, total } = useCartStore();
  return <span>{items.length} items — ${total}</span>;
}
```

Redux Toolkit — Enterprise Scale



Redux: unidirectional data flow — actions → reducers → store → UI

```
import { createSlice, configureStore } from '@reduxjs/toolkit';
import { useSelector, useDispatch } from 'react-redux';

// Slice — reducer + actions in one
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0 },
  reducers: {
    addItem:   (state, action) => { state.items.push(action.payload); }, // Immer!
    removeItem:(state, action) => {
      state.items = state.items.filter(i => i.id !== action.payload);
    },
    clearCart: (state) => { state.items = []; state.total = 0; },
  },
});

export const { addItem, removeItem, clearCart } = cartSlice.actions;

// Store
const store = configureStore({ reducer: { cart: cartSlice.reducer } });

// Component
function CartButton({ product }) {
  const dispatch = useDispatch();
  const count    = useSelector(state => state.cart.items.length);
  return (
    <button onClick={() => dispatch(addItem(product))}>
      Add to Cart ({count})
    </button>
  );
}
```

CHAPTER 11 — FORMS & VALIDATION

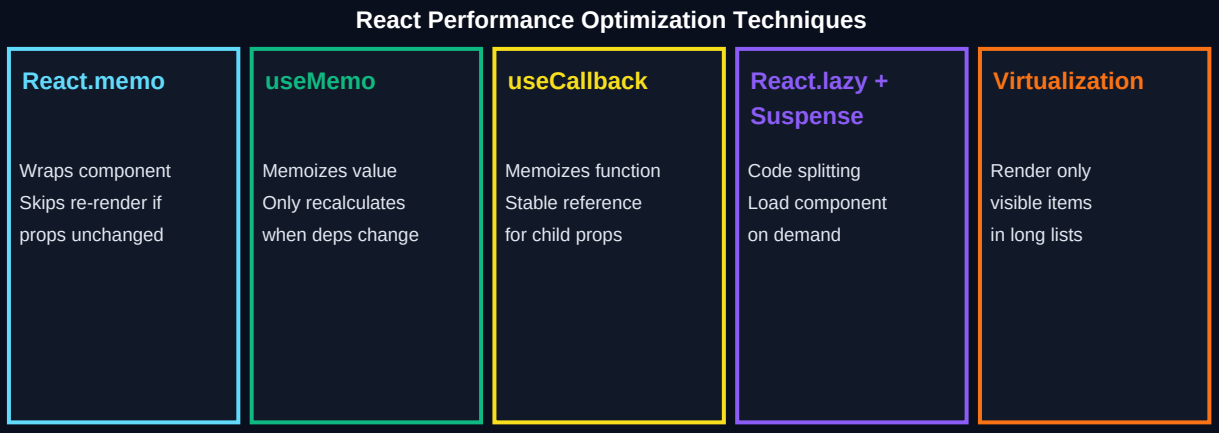
React Hook Form + Zod

React Hook Form avoids unnecessary re-renders by using uncontrolled inputs under the hood. Zod provides type-safe schema validation that integrates with TypeScript. Together they're the industry standard for production forms.

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

// 1. Define validation schema with Zod
const registerSchema = z.object({
  name:     z.string().min(2, 'Name too short').max(50),
  email:    z.string().email('Invalid email address'),
  password: z.string()
             .min(8, 'At least 8 characters')
             .regex(/[A-Z]/, 'Needs uppercase')
             .regex(/[0-9]/, 'Needs a number'),
  age:      z.number().min(18, 'Must be 18+').max(120),
});

type RegisterData = z.infer<typeof registerSchema>; // TypeScript type!

// 2. Hook into the form
function RegisterForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<RegisterData>({ resolver: zodResolver(registerSchema) });

  const onSubmit = async (data: RegisterData) => {
    await createUser(data); // data is fully typed & validated!
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('email')} placeholder='Email' />
      {errors.email && <p>{errors.email.message}</p>}

      <input {...register('password')} type='password' />
      {errors.password && <p>{errors.password.message}</p>}

      <button disabled={isSubmitting}>
        {isSubmitting ? 'Saving...' : 'Register'}
      </button>
    </form>
  );
}
```

CHAPTER 12 — PERFORMANCE OPTIMIZATION

React Performance Toolkit



Performance: prevent unnecessary re-renders, split bundles, virtualize long lists

```
// React.memo — skip re-render if props unchanged
const UserCard = React.memo(function UserCard({ user }) {
  console.log('render'); // Only logs when user actually changes
  return <div>{user.name}</div>;
});

// useMemo — memoize expensive calculation
function Dashboard({ orders }) {
  // Only recalculates when orders changes
  const stats = useMemo(() => ({
    total:   orders.reduce((s, o) => s + o.amount, 0),
    average: orders.reduce((s, o) => s + o.amount, 0) / orders.length,
    top:     orders.sort((a,b) => b.amount - a.amount)[0],
  }), [orders]);
  return <StatsPanel stats={stats} />;
}

// Code Splitting — load component only when needed
const HeavyChart = React.lazy(() => import('./HeavyChart'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      {showChart && <HeavyChart />}  {/* downloaded on demand */}
    </Suspense>
  );
}
```

Optimization	What It Prevents	When to Use	Cost
React.memo	Child re-render with same props	Pure UI components	Shallow comparison overhead
useMemo	Expensive recalculation	Heavy computations	Memory + compare overhead
useCallback	Function recreation every render	Passed to memoized child	Memory overhead
React.lazy	Loading unused JS upfront	Large components/routes	Async complexity
Virtualization	Rendering off-screen list items	Lists > 100 items	Library dependency

CHAPTER 13 — INTRODUCTION TO NEXT.JS

Next.js — The React Framework

Next.js is built on top of React and adds everything React lacks for production: file-based routing, server-side rendering, API routes, image optimization, and more. It's the most popular React framework used by Vercel, Netflix, TikTok, and thousands of companies.

Feature	React (SPA)	Next.js
Routing	Manual (React Router)	File-based (automatic)
Rendering	Client-side only	CSR + SSR + SSG + ISR
SEO	Poor (JS-dependent)	Excellent (pre-rendered HTML)
API Backend	Separate backend needed	Built-in Route Handlers
Images	Manual optimization	<Image /> auto-optimizes
Fonts	Manual loading	next/font auto-optimizes
Config	Webpack/Vite manual	Zero-config with next.config
Deploy	Any static host	Vercel (native) or any Node host

app/ (App Router)

```
layout.tsx        Root layout
page.tsx          / route
about/            folder = route
  page.tsx        /about route
blog/
  [slug]/         dynamic segment
    page.tsx      /blog/:slug
api/
  users/
    route.ts      API endpoint
```

Server vs Client

Server Component

`'use server' (default)`

No JS bundle sent

Fetch data directly

Access DB, secrets

Client Component

`'use client'`

useState/useEffect

Event handlers

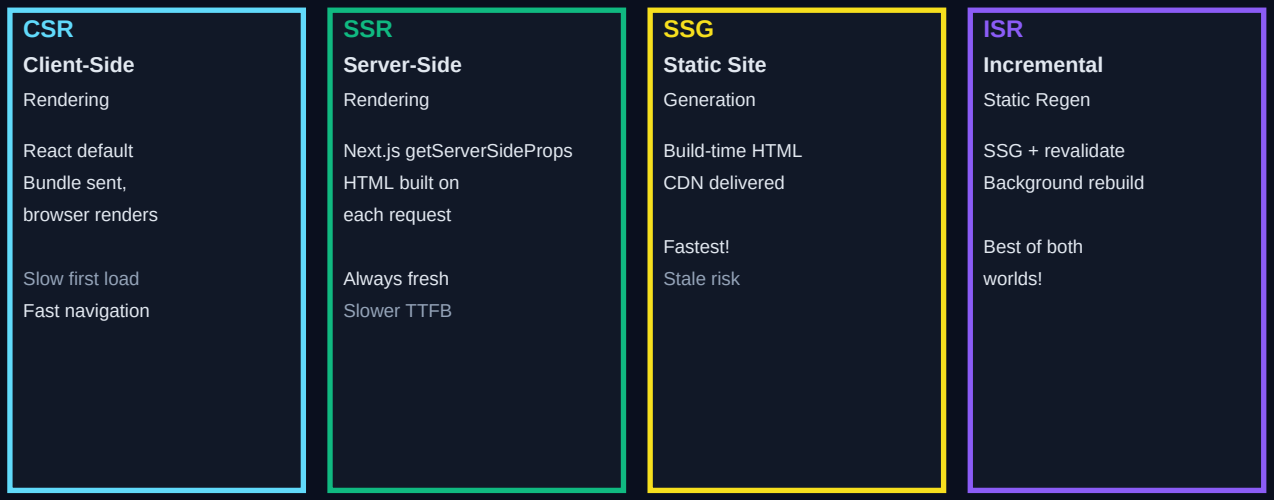
Browser APIs

App Router: folder = route, layout.tsx = shared wrapper, page.tsx = route handler

```
// app/layout.tsx — Root layout (wraps ALL pages)
export default function RootLayout({ children }) {
  return (
    <html lang='en'>
      <body>
        <Navbar />
        <main>{children}</main>
        <Footer />
      </body>
    </html>
  );
}

// app/blog/[slug]/page.tsx — Dynamic route
export default async function BlogPost({ params }) {
  const post = await getPost(params.slug); // Server Component: direct DB access!
  return <article>{post.content}</article>;
}

// Generate static paths at build time
export async function generateStaticParams() {
  const posts = await getAllSlugs();
  return posts.map(p => ({ slug: p.slug }));
}
```

CHAPTER 14 — RENDERING STRATEGIES IN NEXT.JS

CSR vs SSR vs SSG vs ISR



Rendering strategies: each trades off freshness, performance, and complexity

Strategy	When HTML Built	Data Freshness	Best For
CSR	In browser (runtime)	Always fresh	Dashboards, auth-gated pages
SSR	On each request	Always fresh	Product pages, user-specific
SSG	At build time	Stale (rebuild needed)	Blogs, docs, marketing pages
ISR	Build + background	Fresh after revalidate	E-commerce, news sites

```
// SSR — fetch on every request (Next.js App Router)
// Server Components fetch by default — no extra config!
async function ProductPage({ params }) {
  const product = await fetchProduct(params.id); // runs on server, fresh each req
  return <ProductView product={product} />;
}

// SSG — static at build time
async function BlogPage({ params }) {
  const post = await getPost(params.slug);
  return <BlogPost post={post} />;
}
export async function generateStaticParams() {
  return [{ slug: 'intro' }, { slug: 'advanced' }];
}

// ISR — revalidate every N seconds
async function PricingPage() {
  const prices = await fetch('/api/prices', {
    next: { revalidate: 3600 }, // revalidate every 1 hour
  }).then(r => r.json());
  return <PricingTable prices={prices} />;
}
```

CHAPTER 15 — APIs & SERVER ACTIONS

Route Handlers (API Routes)

```
// app/api/users/route.ts
import { NextRequest, NextResponse } from 'next/server';

// GET /api/users
export async function GET(req: NextRequest) {
  const { searchParams } = req.nextUrl;
  const page = searchParams.get('page') ?? '1';
  const users = await db.user.findMany({ skip: (Number(page)-1)*10, take: 10 });
  return NextResponse.json({ users, page });
}

// POST /api/users
export async function POST(req: NextRequest) {
  const body = await req.json();
  const result = userSchema.safeParse(body);
  if (!result.success) return NextResponse.json({ error: result.error }, { status: 422 });
  const user = await db.user.create({ data: result.data });
  return NextResponse.json(user, { status: 201 });
}

// app/api/users/[id]/route.ts — Dynamic API route
export async function DELETE(req: NextRequest, { params }) {
  await db.user.delete({ where: { id: params.id } });
  return NextResponse.json({ deleted: true });
}
```

Server Actions — Next.js 14+

Server Actions let you run server-side code DIRECTLY from a component — no API route needed. They're perfect for form submissions, mutations, and any server operation triggered from UI.

```
// actions/user.ts — Server Action
'use server'; // This entire file runs on the server

import { revalidatePath } from 'next/cache';

export async function createUser(formData: FormData) {
  const name  = formData.get('name') as string;
  const email = formData.get('email') as string;

  // Directly access DB — no HTTP request needed!
  await db.user.create({ data: { name, email } });
  revalidatePath('/users'); // refresh the users page cache
}

// component/UserForm.tsx — Uses Server Action
import { createUser } from '@/actions/user';

export function UserForm() {
  return (
    // action prop accepts a Server Action!
    <form action={createUser}>
      <input name='name'  placeholder='Name' />
      <input name='email' placeholder='Email' />
      <button type='submit'>Create</button>
    </form>
  );
}

// Can also call from event handlers:
// const result = await createUser(formData);
```

EPISODE 2 COMPLETE — WHAT YOU'VE LEARNED

✓ **Why React Exists**	Virtual DOM, declarative UI, SPA architecture
✓ **Components & Props**	Functional components, prop flow, list rendering, keys
✓ **useState & Re-rendering**	State cycle, functional updates, immutable patterns
✓ **useEffect Lifecycle**	Mount/update/unmount, dependency array, cleanup
✓ **Event Handling**	Controlled inputs, form validation, conditional rendering
✓ **Component Architecture**	Smart/dumb split, lifting state, Context API
✓ **All Core Hooks**	useRef, useMemo, useCallback, custom hooks, rules
✓ **React Router v6**	Route definitions, dynamic routes, useNavigate
✓ **TanStack Query**	useQuery, useMutation, caching, background refetch
✓ **Zustand & Redux Toolkit**	Store setup, actions, selectors, when to use each
✓ **Forms with RHF + Zod**	Schema validation, TypeScript integration, error display
✓ **Performance Optimization**	React.memo, useMemo, useCallback, lazy loading
✓ **Next.js App Router**	File routing, layouts, Server vs Client Components
✓ **Rendering Strategies**	CSR, SSR, SSG, ISR — when and why to use each
✓ **APIs & Server Actions**	Route handlers, Server Actions, DB integration

What's Next — Episode 3

Episode 3 covers Node.js, MongoDB & Backend Architecture:

→ Node.js server, NPM, Express.js framework
→ REST API design, MongoDB & Mongoose ORM
→ Authentication (JWT, bcrypt, Passport.js)
→ Middleware, file uploads, caching with Redis
→ WebSockets, real-time apps, production structure